PROYECTO ESCALADA

Guía técnica completa de la aplicación

Laravel 12 | PHP 8.2 | MySQL | Bootstrap 5 | Alpine.js

Plataforma de gestión de competiciones de escalada para la Federación Andaluza. Este documento explica en detalle cómo funciona la aplicación y los conceptos de Laravel utilizados.

Generado el 17 de Marzo de 2026

Índice de contenidos

1. 1. ¿Qué es Laravel? El framework MVC
2. 2. Estructura de archivos del proyecto
3. 3. La base de datos: migraciones y Eloquent ORM
4. 4. Los Modelos: el corazón de los datos
5. 5. Las Rutas: cómo llegan las peticiones
6. 6. Los Controladores: lógica de negocio
7. 7. Las Vistas Blade: el HTML dinámico
8. 8. Autenticación con Laravel Breeze
9. 9. Sistema de roles y middleware
10. 10. Flujo completo: ejemplo de una inscripción
11. 11. Frontend: Bootstrap 5 y Alpine.js
12. 12. Seeders: datos de prueba
13. 13. Diagrama completo de la aplicación

¿Qué es Laravel? El framework MVC

01

El patrón de arquitectura que organiza toda la aplicación

¿Qué es un framework?

Un **framework** es un conjunto de herramientas, convenciones y código ya escrito que te ahorra tener que construir todo desde cero. Laravel es el framework PHP más popular del mundo: te proporciona un sistema de rutas, acceso a base de datos, autenticación, generación de HTML y muchísimo más, para que tú te concentres solo en la lógica de tu aplicación.

El patrón MVC (Modelo - Vista - Controlador)

Laravel sigue el patrón **MVC**, que divide la aplicación en tres capas con responsabilidades bien separadas:

Capa	¿Qué hace?	En nuestro proyecto
Modelo (M)	Gestiona los datos y la base de datos	`app/Models/Competicion.php`, `User.php`, etc.
Vista (V)	Genera el HTML que ve el usuario	`resources/views/` (archivos `.blade.php`)
Controlador (C)	Recibe la petición, usa el modelo y devuelve la vista	`app/Http/Controllers/`

El ciclo de una petición HTTP en Laravel

Cuando el usuario hace clic en un enlace o envía un formulario, esto es lo que ocurre internamente:

1. Navegador envía petición → 2. Laravel recibe en index.php → 3. Middleware (¿auth? ¿rol?) → 4. Router busca la ruta →
5. Controlador ejecuta lógica → 6. Modelo consulta BD → 7. Vista Blade genera HTML → 8. Respuesta al navegador

💡 **Ejemplo real:** Cuando el admin visita `/admin/pruebas`, Laravel verifica que esté autenticado y que tenga rol admin (middleware), busca la ruta en `web.php`, ejecuta

> `AdminController@pruebas()` , que consulta las competiciones en la BD, y devuelve la vista `admin/pruebas.blade.php` con los datos.

¿Por qué PHP y no otro lenguaje?

PHP fue diseñado específicamente para la web. Laravel añade una capa de elegancia sobre PHP haciendo que el código sea más limpio, organizado y fácil de mantener. La versión que usamos, **PHP 8.2**, incluye características modernas como tipado estricto, enums, fibers y mucho más.

Estructura de archivos del proyecto

02

Dónde está cada cosa y por qué

Un proyecto Laravel tiene una estructura de carpetas estandarizada. Aprenderla es esencial para moverte por el código:

```
Escalada/
│
├── app/                        ← El corazón de la aplicación
│   ├── Http/
│   │   ├── Controllers/        ← Los controladores (lógica de negocio)
│   │   │   ├── AdminController.php
│   │   │   ├── ArbitroController.php
│   │   │   ├── CompeticionController.php
│   │   │   ├── CopaController.php
│   │   │   ├── EntrenadorController.php
│   │   │   ├── InscripcionController.php
│   │   │   ├── NotificacionController.php
│   │   │   └── UbicacionController.php
│   │   └── Middleware/
│   │       └── CheckRol.php    ← Verifica el rol del usuario
│   ├── Models/                 ← Los modelos (representación de tablas BD)
│   │   ├── User.php
│   │   ├── Competicion.php
│   │   ├── Copa.php
│   │   ├── Ubicacion.php
│   │   ├── Inscripcion.php
│   │   └── LicenciaValidacion.php
│   └── Notifications/          ← Notificaciones automáticas
│
├── database/
│   ├── migrations/             ← Definición de las tablas BD
│   ├── seeders/                ← Datos de prueba
│   └── factories/              ← Generadores de datos falsos
│
├── resources/
│   ├── views/                  ← Las vistas HTML (Blade)
│   │   ├── layouts/
│   │   │   └── app.blade.php   ← Layout principal (navbar, etc.)
│   │   ├── admin/              ← Vistas del panel admin
│   │   ├── arbitro/            ← Vistas del árbitro
│   │   ├── dashboard/          ← Dashboards por rol
│   │   └── components/         ← Componentes reutilizables
│   └── js/                     ← JavaScript (gestionado por Vite)
│
├── routes/
│   └── web.php                 ← TODAS las rutas de la aplicación
```

```
│
├── public/
│   └── index.php              ← Punto de entrada de TODAS las peticiones
│
├── .env                       ← Configuración del entorno (BD, mail, etc.)
├── composer.json              ← Dependencias PHP
└── package.json               ← Dependencias JavaScript
```

> ⚠️ **Importante:** El único archivo que el servidor web "ve" directamente es `public/index.php`. Todas las peticiones pasan por ahí. El resto de carpetas (`app/`, `routes/`, etc.) están fuera del alcance directo del navegador, lo que es una medida de seguridad fundamental.

El archivo .env

El archivo `.env` (environment) contiene la configuración sensible del entorno: credenciales de base de datos, claves de API, configuración de correo, etc. **Nunca se sube a Git** por razones de seguridad.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=escalada
DB_USERNAME=root
DB_PASSWORD=

APP_NAME=Escalada
APP_URL=http://localhost:8000
APP_LOCALE=es
```

La base de datos: migraciones y Eloquent ORM

03

Cómo Laravel gestiona la estructura y los datos de MySQL

¿Qué son las migraciones?

Las **migraciones** son archivos PHP que describen la estructura de las tablas de la base de datos. En lugar de escribir SQL directamente, describes las tablas en PHP y Laravel las crea automáticamente. Esto permite que cualquier desarrollador reconstruya la BD con un solo comando: `php artisan migrate`.

> ✅ **Ventaja clave:** Si añades una columna nueva a una tabla, creas una migración nueva y todos los demás miembros del equipo la reciben al hacer `git pull` + `php artisan migrate`. No hay que compartir scripts SQL manualmente.

Ejemplo: Migración de competiciones

```
// database/migrations/2026_02_16_150333_create_competicions_table.php
Schema::create('competicions', function (Blueprint $table) {
    $table->id();                          // Columna id autoincremental

    $table->foreignId('copa_id')           // Clave foránea a tabla "copas"
          ->nullable()                     // Puede ser null (sin copa)
          ->constrained('copas')           // Referencia a copas.id
          ->nullOnDelete();                // Si se borra la copa → null (no cas

    $table->foreignId('arbitro_id')        // El árbitro asignado
          ->nullable()
          ->constrained('users')           // Referencia a users.id
          ->nullOnDelete();

    $table->foreignId('ubicacion_id')      // Rocódromo donde se celebra
          ->constrained('ubicacions')
          ->cascadeOnDelete();             // Si se borra el rocódromo → borra es

    $table->string('name');                // Nombre de la competición
    $table->string('provincia');           // Provincia
    $table->dateTime('fecha_realizacion');// Fecha y hora de inicio
    $table->dateTime('fecha_fin')->nullable(); // Fecha fin (puede ser null)
    $table->string('tipo');                // bloque / dificultad / velocidad
    $table->boolean('campeonato')->default(false); // ¿Es campeonato?
    $table->json('categorias')->nullable(); // Array JSON de categorías
```

```
    $table->timestamps();                // created_at y updated_at automáticos
});
```

Diagrama de la base de datos

Tabla	Columnas principales	Se relaciona con
`users`	name, email, dni, rol, provincia, talla, genero	competicions, inscripciones, entrenador_competidor
`copas`	name, tipo, temporada	competicions
`ubicacions`	name, provincia, direccion, alto, ancho, n_lineas	competicions
`competicions`	name, tipo, fecha_realizacion, fecha_fin, campeonato, categorias	copas, users (árbitro), ubicacions, inscripciones
`competicions_users`	user_id, competicion_id, tipoDato, dato	users, competicions (tabla pivote)
`inscripciones`	estado, licencia_path, pago_path, categoria, licencia_estado, pago_estado	users, competicions
`licencia_validaciones`	user_id, competicion_id, tipo, valida_hasta	users, competicions
`entrenador_competidor`	entrenador_id, competidor_id, estado	users (doble)
`notifications`	type, notifiable_id, data, read_at	users (polimórfica)

¿Qué es Eloquent ORM?

ORM significa *Object-Relational Mapping*. Es un sistema que te permite trabajar con la base de datos usando objetos PHP en lugar de SQL puro. Eloquent es el ORM de Laravel.

```
// Sin Eloquent (SQL puro — tedioso):
$result = DB::select("SELECT * FROM competicions WHERE id = 5");

// Con Eloquent (mucho más limpio):
$competicion = Competicion::find(5);
echo $competicion->name;          // Accedes a columnas como propiedades
```

```
echo $competicion->copa->name;    // ¡Accedes a relaciones automáticamente!
echo $competicion->inscripciones->count(); // Cuenta relaciones
```

💡 **Concepto clave:** En Eloquent, cada tabla de la base de datos corresponde a una clase PHP (Modelo). Cada fila de la tabla corresponde a un objeto de esa clase. Modificar el objeto y llamar a `save()` actualiza la fila en la BD automáticamente.

Los Modelos: el corazón de los datos

04

Cómo representamos y relacionamos las entidades del dominio

Modelo User — El usuario del sistema

Es el modelo más complejo. Un `User` puede tener múltiples roles con comportamientos distintos:

```
class User extends Authenticatable
{
    protected $fillable = [
        'name', 'email', 'password', 'dni',
        'fecha_nacimiento', 'provincia', 'talla', 'genero',
        'rol',   // 'competidor' | 'entrenador' | 'arbitro' | 'admin'
    ];

    // $hidden oculta estos campos cuando convertimos a JSON/array
    protected $hidden = ['password', 'remember_token'];

    // Métodos de comprobación de rol
    public function isAdmin():      bool { return $this->nivelRol() >= 4; }
    public function isArbitro():    bool { return $this->nivelRol() >= 3; }
    public function isEntrenador(): bool { return $this->nivelRol() >= 2; }
    public function isCompetidor(): bool { return $this->nivelRol() >= 1; }

    // Nivel numérico del rol (para jerarquía)
    public function nivelRol(): int {
        return match($this->rol) {
            'admin'      => 4,
            'arbitro'    => 3,
            'entrenador' => 2,
            default      => 1,  // competidor
        };
    }
}
```

Relaciones del modelo User

Las **relaciones** son la parte más potente de Eloquent. Definen cómo se conectan los modelos entre sí:

Método	Tipo	Descripción
`competiciones()`	BelongsToMany	Competiciones en las que participa (tabla pivote

		`competicions_users`)
`competicionesArbitradas()`	HasMany	Competiciones donde está asignado como árbitro
`inscripciones()`	HasMany	Todas las inscripciones del usuario
`competidoresAceptados()`	BelongsToMany	Competidores bajo su tutela (estado = aceptado)
`competidoresPendientes()`	BelongsToMany	Solicitudes de vínculo pendientes
`entrenadores()`	BelongsToMany	Entrenadores que lo supervisan (relación inversa)

Los tipos de relaciones en Eloquent

HasMany (uno a muchos)

```
// Una Copa tiene MUCHAS Competiciones
class Copa extends Model {
    public function competiciones(): HasMany {
        return $this->hasMany(Competicion::class, 'copa_id');
    }
}
// Uso:
$copa = Copa::find(1);
$competiciones = $copa->competiciones; // Devuelve una Collection de Competicion
```

BelongsTo (muchos a uno, inversa)

```
// Una Competicion PERTENECE a una Copa
class Competicion extends Model {
    public function copa(): BelongsTo {
        return $this->belongsTo(Copa::class, 'copa_id');
    }
}
// Uso:
$competicion = Competicion::find(1);
echo $competicion->copa->name; // "Copa de Bloque 2026"
echo $competicion->copa?->name; // Con "?" por si copa_id es null
```

BelongsToMany (muchos a muchos)

```
// Un User puede estar en MUCHAS Competiciones,
// y una Competicion puede tener MUCHOS Users
```

```
// Se necesita tabla intermedia: competicions_users
class User extends Model {
    public function competiciones(): BelongsToMany {
        return $this->belongsToMany(
            Competicion::class,
            'competicions_users',   // nombre de la tabla pivote
            'user_id',              // FK de este modelo en la pivote
            'competicion_id'        // FK del otro modelo en la pivote
        )->withPivot('tipoDato', 'dato') // columnas extra de la pivote
         ->withTimestamps();
    }
}
```

El modelo Inscripcion — lógica de negocio avanzada

El modelo `Inscripcion` tiene lógica de negocio propia, algo muy común en Laravel:

```
public function recalcularEstado(): void
{
    // Si algún documento es inválido → rechazada
    if ($this->licencia_estado === 'no_valida' || $this->pago_estado === 'no_val
        $this->estado = 'rechazada';
    }
    // Si ambos documentos son válidos → aprobada
    elseif (
        in_array($this->licencia_estado, ['valida', 'valida_dia']) &&
        in_array($this->pago_estado,     ['valida', 'valida_dia'])
    ) {
        $this->estado = 'aprobada';
    }
    // Si no → pendiente
    else {
        $this->estado = 'pendiente';
    }
    $this->save(); // Guarda en la BD
}

// Calcula la categoría automáticamente según la edad y género del competidor
public static function calcularCategoria(User $user): string
{
    $edad = Carbon::parse($user->fecha_nacimiento)->age;
    $cat  = match(true) {
        $edad <= 8  => 'U9',
        $edad <= 10 => 'U11',
        $edad <= 12 => 'U13',
        $edad <= 14 => 'U15',
        $edad <= 16 => 'U17',
        $edad <= 18 => 'U19',
        $edad <= 34 => 'Absoluta',
        default     => 'Veterana',
    };
    $genero = match($user->genero) {
        'M' => 'Masculino',
```

```
        'F' => 'Femenino',
        default => 'Otro',
    };
    return "$genero $cat"; // Ej: "Femenino U15"
}
```

$fillable y la protección de asignación masiva

⚠️ **Seguridad:** En Laravel, `$fillable` define qué campos pueden asignarse masivamente. Si un usuario malicioso envía `rol=admin` en un formulario, Laravel lo ignorará si `rol` no está en `$fillable`. Esto previene ataques de *mass assignment*.

Las Rutas: cómo llegan las peticiones

05

El archivo web.php — el mapa de toda la aplicación

¿Qué es una ruta?

Una **ruta** asocia una URL y un método HTTP (GET, POST, PATCH, DELETE) con una acción específica del controlador. Todo esto se define en `routes/web.php`.

```
// GET /admin/copas → ejecuta AdminController::copas() → devuelve vista
Route::get('/admin/copas', [AdminController::class, 'copas'])->name('admin.copas

// POST /admin/copas → ejecuta CopaController::store() → guarda nueva copa
Route::post('/admin/copas', [CopaController::class, 'store'])->name('admin.copas

// PATCH /admin/copas/{copa} → ejecuta CopaController::update() → edita copa
Route::patch('/admin/copas/{copa}', [CopaController::class, 'update'])->name('ad

// DELETE /admin/copas/{copa} → ejecuta CopaController::destroy() → borra copa
Route::delete('/admin/copas/{copa}', [CopaController::class, 'destroy'])->name(
```

💡 **Los métodos HTTP:** Los navegadores solo pueden enviar GET y POST. Para PATCH y DELETE, Laravel usa un campo oculto `_method` en el formulario. El Blade `@method('PATCH')` lo genera automáticamente.

Rutas con parámetros: Route Model Binding

Cuando una ruta tiene `{copa}` o `{competicion}`, Laravel automáticamente busca el registro correspondiente en la BD y lo inyecta en el controlador. A esto se llama **Route Model Binding**.

```
// Ruta:
Route::patch('/admin/copas/{copa}', [CopaController::class, 'update']);

// Controlador — Laravel inyecta automáticamente el objeto Copa:
public function update(Request $request, Copa $copa)
{
    // $copa ya es el objeto Copa con id = el número de la URL
    // Si no existe → Laravel devuelve automáticamente un 404
    $copa->update(['name' => $request->name]);
}
```

Grupos de rutas con middleware y prefijos

En vez de repetir el middleware y prefijo en cada ruta, se agrupan:

```
// Todas las rutas admin comparten: autenticación + rol admin + prefijo /admin
Route::middleware(['auth', 'rol:admin'])->prefix('admin')->name('admin.')->grou
    Route::get('/pruebas',     [AdminController::class, 'pruebas'])->name('prueba
    Route::get('/copas',       [AdminController::class, 'copas'])->name('copas'),
    Route::get('/usuarios',    [AdminController::class, 'usuarios'])->name('usua
    Route::get('/rocodromos', [AdminController::class, 'rocodromos'])->name('roc
    // ...más rutas
});
// Resultado: ruta 'admin.copas' → URL /admin/copas → requiere auth + rol:admin
```

Rutas con nombre (named routes)

El `->name('admin.copas')` asigna un nombre a la ruta. Esto es fundamental porque en las vistas y controladores **nunca escribimos URLs hardcodeadas**; en su lugar usamos el nombre:

```
// En una vista Blade — genera la URL correcta automáticamente:
<a href="{{ route('admin.copas') }}">Ver copas</a>
<form action="{{ route('admin.copas.update', $copa->id) }}" method="POST">

// En un controlador — redirige a una ruta con nombre:
return redirect()->route('admin.pruebas')->with('status', 'Copa creada.');
```

> ✅ **Ventaja:** Si cambias la URL de una ruta (de `/admin/copas` a `/panel/copas`), solo tienes que cambiarlo en `web.php`. Todas las vistas que usen `route('admin.copas')` seguirán funcionando automáticamente.

Resumen de todas las rutas del proyecto

Grupo	Prefijo	Middleware	Rutas principales
Dashboard	/dashboard	auth	Redirige según rol
Admin	/admin	auth, rol:admin	CRUD competiciones, copas, usuarios, rocódromos
Árbitro	/arbitro	auth, rol:arbitro	Panel, validar inscripciones, gestionar equipo

Entrenador	/entrenador	auth, rol:entrenador	Solicitar vínculos, inscribir competidores
Inscripciones	/inscripciones, /competiciones	auth	Ver, crear inscripción, subir documentos
Notificaciones	/notificaciones	auth	Aceptar/rechazar/desvincular
Perfil	/profile	auth	Ver, editar, eliminar cuenta
Auth	/login, /register...	guest	Login, registro, reset password

Los Controladores: lógica de negocio

06

Cómo procesa Laravel cada petición

¿Qué hace un controlador?

El controlador es el intermediario entre las rutas y los modelos/vistas. Recibe la petición HTTP, valida los datos, consulta o modifica la BD usando los modelos, y devuelve una respuesta (normalmente una vista o una redirección).

AdminController — Panel de administración

```
// Método para listar y filtrar competiciones (pruebas):
public function pruebas(Request $request)
{
    $filtro  = $request->get('filtro', 'proximas');
    $copaId  = $request->get('copa_id', '');

    // Construimos la query de forma dinámica según los filtros
    $query = Competicion::with('copa', 'ubicacion', 'arbitro')
                        ->withCount('inscripciones');

    // Aplicamos filtro de fecha
    if ($filtro === 'proximas') {
        $query->where('fecha_realizacion', '>=', now());
    } elseif ($filtro === 'este_año') {
        $query->whereYear('fecha_realizacion', now()->year);
    }

    // Aplicamos filtro de copa
    if ($copaId === 'sin_copa') {
        $query->whereNull('copa_id');
    } elseif ($copaId) {
        $query->where('copa_id', $copaId);
    }

    $competiciones = $query->orderBy('fecha_realizacion')->get();

    // Datos necesarios para los formularios
    $copas     = Copa::orderBy('temporada', 'desc')->get();
    $ubicaciones = Ubicacion::orderBy('name')->get();
    $arbitros  = User::where('rol', 'arbitro')->orWhere('rol', 'admin')->get();

    // Devolvemos la vista con todos los datos necesarios
```

```
    return view('admin.pruebas', compact('competiciones', 'copas', 'ubicaciones
}
```

CompeticionController — CRUD de competiciones

```
public function store(Request $request)
{
    // 1. VALIDACIÓN: Laravel valida automáticamente y devuelve errores si falla
    $request->validate([
        'name'              => 'required|string|max:150',
        'tipo'              => 'required|in:bloque,dificultad,velocidad',
        'fecha_realizacion' => 'required|date',
        'fecha_fin'         => 'nullable|date|after_or_equal:fecha_realizacion',
        'provincia'         => 'required|string',
        'ubicacion_id'      => 'required|exists:ubicacions,id', // debe existir
        'copa_id'           => 'nullable|exists:copas,id',
        'arbitro_id'        => 'nullable|exists:users,id',
        'categorias'        => 'nullable|array',
        'categorias.*'      => 'in:U9,U11,U13,U15,U17,U19,Absoluta,Veterana,Pro
    ]);

    // 2. CREACIÓN: Eloquent crea el registro en la BD
    Competicion::create([
        'name'              => $request->name,
        'tipo'              => $request->tipo,
        'fecha_realizacion' => $request->fecha_realizacion,
        'fecha_fin'         => $request->fecha_fin ?: null,
        'provincia'         => $request->provincia,
        'ubicacion_id'      => $request->ubicacion_id,
        'copa_id'           => $request->copa_id ?: null,
        'arbitro_id'        => $request->arbitro_id ?: null,
        'campeonato'        => false,
        'categorias'        => $request->categorias ?? [],
    ]);

    // 3. REDIRECCIÓN con mensaje flash (se muestra una vez)
    return back()->with('status', "Prueba «{$request->name}» creada correctamen
}
```

> 💡 **Mensajes flash:** `->with('status', 'mensaje')` almacena el mensaje en la sesión durante UNA sola petición. En la vista, `session('status')` lo recupera. Es el patrón estándar para mostrar confirmaciones tras formularios.

ArbitroController — Validación de inscripciones

El árbitro puede verificar documentos de cada inscripción. Este es el método más complejo del proyecto:

```
public function validarLicencia(Request $request, Inscripcion $inscripcion)
{
    // Verificamos que este usuario es el árbitro asignado a esta competición
    $this->checkArbitroPorCompeticion($inscripcion->competicion_id);

    $request->validate([
        'tipo'     => 'required|in:licencia,pago',
        'decision' => 'required|in:valida,valida_dia,no_valida',
        'motivo'   => 'required_if:decision,no_valida|nullable|string|max:500',
    ]);

    if ($request->tipo === 'licencia') {
        $inscripcion->licencia_estado = $request->decision;
        $inscripcion->licencia_motivo = $request->decision === 'no_valida'
            ? $request->motivo : null;

        // Si la licencia es válida, registramos en LicenciaValidacion
        // para que no haya que re-validarla en otras competiciones del mismo a
        if ($request->decision !== 'no_valida') {
            $validaHasta = $request->decision === 'valida'
                ? now()->endOfYear()->toDateString()       // válida hasta 31/1
                : $inscripcion->competicion->fecha_realizacion->toDateString();

            LicenciaValidacion::updateOrCreate(
                ['user_id' => $inscripcion->user_id, 'competicion_id' => $inscr
                ['validada_por' => auth()->id(), 'tipo' => $request->decision,
            );
        }
    } else {
        $inscripcion->pago_estado = $request->decision;
        $inscripcion->pago_motivo = $request->decision === 'no_valida' ? $reques
    }

    // Recalcular el estado global de la inscripción
    $estadoAnterior = $inscripcion->estado;
    $inscripcion->recalcularEstado();

    // Notificar al competidor si el estado global cambia
    if ($inscripcion->estado !== $estadoAnterior) {
        $inscripcion->user->notify(new InscripcionActualizadaNotification($insc
    }

    // Responde en JSON (la llamada viene de Alpine.js / fetch)
    return response()->json(['success' => true, 'message' => 'Verificado.']);
}
```

Las Vistas Blade: el HTML dinámico

07

Cómo Laravel genera el HTML que ve el usuario

¿Qué es Blade?

Blade es el motor de plantillas de Laravel. Los archivos terminan en `.blade.php` y mezclan HTML con directivas especiales de Blade que comienzan con `@`. Laravel compila estas plantillas a PHP puro y las cachea para mayor rendimiento.

El layout principal (layouts/app.blade.php)

En lugar de repetir la barra de navegación y el pie en cada página, existe un **layout** que define la estructura común. Todas las vistas del proyecto lo extienden:

```
<!-- resources/views/layouts/app.blade.php -->
<!doctype html>
<html lang="es">
<head>
    <title>@yield('title', 'Escalada')</title>  <!-- Las vistas definen el titul
    <link href="bootstrap.min.css" rel="stylesheet">
</head>
<body>
    <nav class="navbar">...</nav>  <!-- Navbar común a todas las páginas -->

    <main class="container py-4">
        @if (session('status'))  <!-- Mensajes flash de éxito -->
            <div class="alert alert-success">{{ session('status') }}</div>
        @endif

        @yield('content')  <!-- ← AQUÍ va el contenido de cada vista -->
    </main>

    <script src="bootstrap.bundle.min.js"></script>
    <script src="alpinejs/cdn.min.js"></script>
    @yield('scripts')  <!-- Scripts adicionales opcionales -->
</body>
</html>
```

Una vista que extiende el layout:

```
<!-- resources/views/admin/copas.blade.php -->
@extends('layouts.app')         <!-- Hereda el layout -->
```

```
@section('title', 'Admin — Copas')  <!-- Define el título -->

@section('content')             <!-- Define el contenido que va en @yield('conte
<div class="row g-4">
    <div class="col-auto">
        @include('admin.partials.sidebar')  <!-- Incluye el sidebar -->
    </div>
    <div class="col">
        <h4>Copas</h4>
        ...
    </div>
</div>
@endsection
```

Directivas Blade más utilizadas en el proyecto

Directiva	Función	Ejemplo en el proyecto
`{{ $var }}`	Imprime el valor escapando HTML (seguro)	`{{ $copa->name }}`
`{!! $var !!}`	Imprime sin escapar (solo para HTML de confianza)	Raramente usado
`@if / @else / @endif`	Condicionales	`@if($c->campeonato) <badge> @endif`
`@foreach / @endforeach`	Bucles	`@foreach($copas as $copa)`
`@forelse / @empty`	Bucle con caso vacío	Tablas con "No hay datos"
`@auth / @guest`	Condicional de autenticación	Mostrar/ocultar botón de login
`@csrf`	Token de seguridad en formularios	Todos los formularios POST
`@method('PATCH')`	Simula métodos HTTP no soportados	Formularios de edición
`@selected($cond)`	Añade `selected` si condición es true	Selects de formularios
`@checked($cond)`	Añade `checked` si condición es true	Checkboxes de categorías
`@include('partial')`	Incluye otra vista	Sidebar del admin
`@extends / @section`	Herencia de layouts	Todas las vistas

Pasar datos a las vistas con compact()

```
// En el controlador — se pasan los datos:
return view('admin.copas', compact('copas', 'filtro'));
// Equivalente a:
return view('admin.copas', ['copas' => $copas, 'filtro' => $filtro]);

// En la vista — se usan directamente:
@foreach($copas as $copa)
    <td>{{ $copa->name }}</td>
    <td>{{ $copa->temporada }}</td>
@endforeach
```

Seguridad automática: XSS prevention

🔒 **Seguridad:** Las llaves dobles `{{ }}` escapan automáticamente los caracteres HTML especiales (`<` , `>` , `"` , etc.), previniendo ataques XSS (Cross-Site Scripting). Si un usuario introduce `<script>alert('hack')</script>` en su nombre, Blade lo mostrará como texto literal, nunca lo ejecutará.

Autenticación con Laravel Breeze

08

Cómo funciona el sistema de login y sesiones

¿Qué es Laravel Breeze?

Laravel Breeze es un starter kit oficial de Laravel que proporciona todo lo necesario para autenticación: login, registro, recuperación de contraseña y verificación de email. Genera las vistas, rutas y controladores listos para usar.

El middleware 'auth'

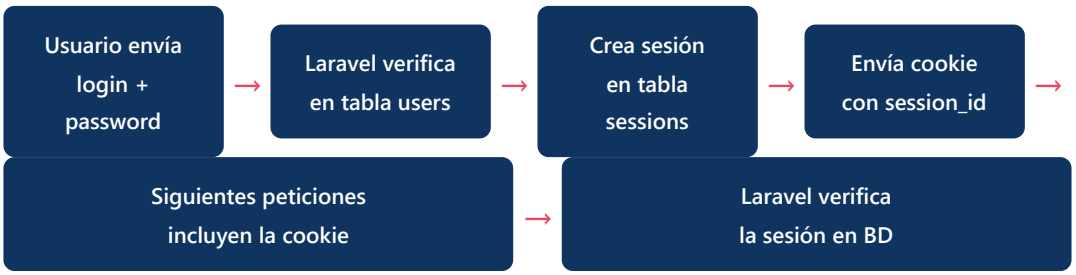
El middleware `auth` es la "puerta" que protege las rutas privadas. Si el usuario no está autenticado, lo redirige al login:

```
// En web.php — esta ruta requiere estar logueado:
Route::get('/dashboard', function () { ... })->middleware(['auth'])->name('dashb

// Si un usuario no autenticado visita /dashboard:
// → Laravel lo redirige automáticamente a /login
```

La sesión: cómo Laravel "recuerda" al usuario

HTTP es un protocolo *sin estado*: cada petición es independiente. Para mantener al usuario logueado entre páginas, Laravel usa **sesiones**. En este proyecto, las sesiones se almacenan en la base de datos (tabla `sessions`):



Acceso al usuario autenticado

```
// En cualquier controlador o vista:
auth()->user()          // Objeto User del usuario actual
auth()->id()            // ID del usuario actual
auth()->check()         // true si está autenticado
auth()->guest()         // true si NO está autenticado

// En una vista Blade:
@auth
    Hola, {{ auth()->user()->name }}
    @if(auth()->user()->isAdmin())
        <a href="/admin">Panel Admin</a>
    @endif
@endauth

@guest
    <a href="/login">Iniciar sesión</a>
@endguest
```

Protección CSRF

CSRF (Cross-Site Request Forgery) es un ataque donde una web maliciosa hace peticiones en nombre del usuario. Laravel lo previene con un token único por sesión.

```
<!-- Todo formulario POST debe incluir @csrf -->
<form method="POST" action="{{ route('admin.copas.store') }}">
    @csrf  <!-- Genera: <input type="hidden" name="_token" value="abc123..."> --
    <input name="name" required>
    <button>Crear</button>
</form>
```

> ⚠️ **Atención:** Si olvidas `@csrf` en un formulario POST, Laravel devolverá un error 419 (Page Expired). Es un error muy común al aprender Laravel.

Hash de contraseñas

Las contraseñas **nunca se guardan en texto plano**. Laravel usa **bcrypt** (o Argon2) para hacer un hash unidireccional:

```
// Al registrarse — guarda el hash, no la contraseña:
'password' => Hash::make($request->password)
// Resultado en BD: "$2y$12$vH5SILdkhlC42So8T9bc..."
```

```
// Al hacer login — compara el hash:
Hash::check($request->password, $user->password) // true o false
```

Sistema de roles y middleware

Cómo se controla quién puede hacer qué

09

Los cuatro roles del sistema

4
Admin
Control total: usuarios, competiciones, copas, rocódromos, árbitros

3
Árbitro
Valida inscripciones y documentos en sus competiciones asignadas

2
Entrenador
Gestiona su equipo e inscribe competidores en competiciones

1
Competidor
Se inscribe en competiciones y sube sus documentos

💡 **Jerarquía:** Un árbitro tiene nivel 3, lo que significa que *también* tiene los permisos de entrenador (nivel 2) y competidor (nivel 1). El middleware verifica `nivelRol() >= nivelRequerido`, no igualdad estricta.

El middleware CheckRol

Un **middleware** es código que se ejecuta antes de que la petición llegue al controlador. El middleware `CheckRol` verifica que el usuario tenga el nivel de rol suficiente:

```
// app/Http/Middleware/CheckRol.php
class CheckRol
{
    public function handle(Request $request, Closure $next, string ...$roles): I
    {
        $user = auth()->user();

        // Mapa de roles a niveles numéricos
        $niveles = ['competidor' => 1, 'entrenador' => 2, 'arbitro' => 3, 'admi

        // Nivel mínimo requerido (el más bajo de los roles pedidos)
        $nivelRequerido = min(array_map(fn($r) => $niveles[$r] ?? 0, $roles));

        // Si el usuario no tiene nivel suficiente → redirige con error
        if (!$user || $user->nivelRol() < $nivelRequerido) {
            return redirect()->route('dashboard')
```

```
                                ->with('error', 'No tienes permiso para acceder aqu
        }

        return $next($request); // Continúa con la petición
    }
}
```

Cómo se registra y usa el middleware:

```
// bootstrap/app.php — registro del middleware con alias:
$middleware->alias(['rol' => CheckRol::class]);

// routes/web.php — uso en grupos de rutas:
Route::middleware(['auth', 'rol:admin'])->group(function () {
    // Solo accesible por admins (nivel 4)
});

Route::middleware(['auth', 'rol:arbitro'])->group(function () {
    // Accesible por árbitros (nivel 3+) y admins (nivel 4)
});
```

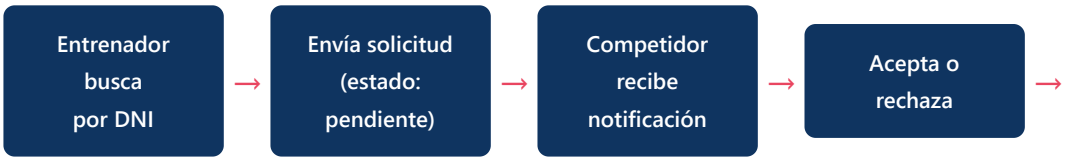
Registro y cambio de roles

Por defecto, los usuarios se registran con rol `competidor`. Solo el admin puede cambiar los roles desde el panel:

```
// AdminController@updateRol
public function updateRol(Request $request, User $user)
{
    $request->validate([
        'rol' => 'required|in:competidor,entrenador,arbitro'
        // Nota: no incluye 'admin' — no se puede hacer admin desde aquí
    ]);
    $user->update(['rol' => $request->rol]);
    return back()->with('status', "Rol de {$user->name} actualizado.");
}
```

El sistema entrenador-competidor

La relación entre entrenador y competidor usa una tabla pivote `entrenador_competidor` con un estado de aprobación, similar a cómo funciona "seguir" en redes sociales:



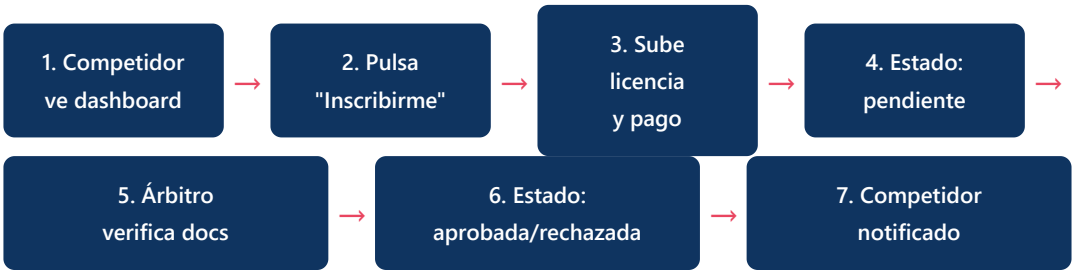
Si acepta:
estado = aceptado

Flujo completo: ejemplo de una inscripción

10

De principio a fin: cómo un competidor se inscribe en una competición

Visión general del proceso



Paso 1 y 2: InscripcionController@store

```
// Cuando el competidor pulsa "Inscribirme":
public function store(Request $request, Competicion $competicion)
{
    // Verifica que no esté ya inscrito
    if ($competicion->inscripciones()->where('user_id', auth()->id())->exists()
        return back()->with('error', 'Ya estás inscrito en esta competición.');
    }

    // Crea la inscripción en estado "pendiente" (sin documentos aún)
    Inscripcion::create([
        'user_id'        => auth()->id(),
        'competicion_id' => $competicion->id,
        'estado'         => 'pendiente',
        // Calcula categoría automáticamente por edad y género
        'categoria'      => Inscripcion::calcularCategoria(auth()->user()),
    ]);

    return back()->with('status', 'Inscripción realizada. Ahora sube tus docume
}
```

Paso 3: Subida de documentos

```
// Cuando sube la licencia federativa:
public function uploadLicencia(Request $request, Competicion $competicion)
{
    $request->validate([
```

```
        'licencia' => 'required|file|mimes:pdf,jpg,jpeg,png|max:5120' // max 5MB
    ]);

    $inscripcion = Inscripcion::where('user_id', auth()->id())
                              ->where('competicion_id', $competicion->id)
                              ->firstOrFail();

    // Laravel guarda el archivo en storage/app/public/inscripciones/
    // y devuelve la ruta relativa
    $path = $request->file('licencia')->store('inscripciones', 'public');

    $inscripcion->update(['licencia_path' => $path]);

    return back()->with('status', 'Licencia subida correctamente.');
}
```

Paso 5: El árbitro verifica un documento

El árbitro accede a `/arbitro/competicion/{id}`, ve la lista de competidores por categoría, abre el modal de verificación y marca la licencia como válida/no válida. La llamada se hace con `fetch()` de JavaScript (Alpine.js):

```
// Alpine.js en la vista — llamada AJAX al servidor:
async verificar(tipo) {
    const resp = await fetch('/arbitro/inscripcion/{{ $ins->id }}/validar', {
        method: 'PATCH',
        headers: {
            'Content-Type': 'application/json',
            'X-CSRF-TOKEN': document.querySelector('meta[name=csrf-token]').cont
        },
        body: JSON.stringify({
            tipo: 'licencia',
            decision: 'valida',   // valida / valida_dia / no_valida
            motivo: null
        })
    });
    const data = await resp.json();
    // Actualiza la UI sin recargar la página
    this.licEstado = data.licencia_estado;
}
```

Paso 6 y 7: Recalcular estado y notificar

Después de que el árbitro valida, el sistema recalcula el estado global y notifica al competidor:

```
// En ArbitroController (simplificado):
$inscripcion->recalcularEstado(); // → 'aprobada' o 'rechazada' o 'pendiente'

// Si el estado cambió a final → notificar al competidor
if (in_array($inscripcion->estado, ['aprobada', 'rechazada'])) {
    $inscripcion->user->notify(
        new InscripcionActualizadaNotification($inscripcion, $motivo)
```

```
    );
}
// La notificación se guarda en la tabla "notifications"
// El competidor la ve en su dashboard la próxima vez que entre
```

Estados de una inscripción

Estado inscripción	licencia_estado	pago_estado	Descripción
pendiente	null	null	Recién inscrito, sin documentos revisados
pendiente	valida	null	Licencia OK, esperando verificar pago
aprobada	valida	valida	Ambos documentos válidos ✅
aprobada	valida_dia	valida	Licencia solo para hoy, pago OK ✅
rechazada	no_valida	—	Licencia inválida ❌
rechazada	—	no_valida	Justificante de pago inválido ❌

Frontend: Bootstrap 5 y Alpine.js

11

Las herramientas que dan vida a la interfaz de usuario

Bootstrap 5 — El sistema de diseño

Bootstrap es un framework CSS que proporciona componentes visuales listos para usar (botones, tarjetas, tablas, modales, formularios...) y un sistema de rejilla responsiva. Se carga desde CDN, sin necesidad de instalar nada:

```
<!-- En layouts/app.blade.php -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bun
```

El sistema de rejilla (Grid)

```
<!-- El layout de admin usa la rejilla de Bootstrap -->
<div class="row g-4">             <!-- fila con gap de 4 unidades -->
    <div class="col-auto">         <!-- columna que se ajusta a su contenido --
        @include('admin.partials.sidebar')
    </div>
    <div class="col">              <!-- columna que ocupa el resto del espacio
        <!-- contenido principal -->
    </div>
</div>
```

Alpine.js — Interactividad sin complejidad

Alpine.js es un framework JavaScript minimalista que añade interactividad directamente en el HTML, sin necesidad de escribir ficheros `.js` separados. Es perfecto para casos de uso simples como modales, toggles, formularios dinámicos.

Ejemplo: El modal de editar Copa

```
<!-- En admin/copas.blade.php -->

<!-- 1. Botón que abre el modal con data-* attributes (Blade renderiza los datos
<button data-bs-toggle="modal" data-bs-target="#modalEditarCopa"
        data-id="{{ $copa->id }}"
        data-name="{{ $copa->name }}"
```

```
        data-tipo="{{ $copa->tipo }}"
        data-temporada="{{ $copa->temporada }}">
    Editar
</button>

<!-- 2. Modal con formulario -->
<div class="modal fade" id="modalEditarCopa">
    <form method="POST" action="" id="editarCopaForm">
        @csrf @method('PATCH')
        <input name="name"      id="editarCopaName">
        <select name="tipo"     id="editarCopaTipo">...</select>
        <input name="temporada" id="editarCopaTemporada">
    </form>
</div>

<!-- 3. JavaScript: Bootstrap dispara el evento 'show.bs.modal' justo antes de a
<script>
document.getElementById('modalEditarCopa').addEventListener('show.bs.modal', fun
    const btn = event.relatedTarget; // El botón que abrió el modal
    // Leemos los data-* del botón y rellenamos el formulario
    document.getElementById('editarCopaForm').action = '/admin/copas/' + btn.dat
    document.getElementById('editarCopaName').value = btn.dataset.name;
    document.getElementById('editarCopaTipo').value = btn.dataset.tipo;
    document.getElementById('editarCopaTemporada').value = btn.dataset.temporada
});
</script>
```

Alpine.js en los modales de validación del árbitro

```
<!-- x-data define el estado reactivo del componente -->
<div class="modal-content" x-data="{
    licDecision: '',           // Decisión tomada para la licencia
    loading: false,            // Spinner de carga
    msg: '',                   // Mensaje de resultado
    licEstado: 'pendiente',    // Estado actual de la licencia

    // Método asíncrono que llama a la API del servidor
    async verificar(tipo) {
        this.loading = true;
        const resp = await fetch('/arbitro/inscripcion/{{ $ins->id }}/validar',
            method: 'PATCH',
            headers: { 'X-CSRF-TOKEN': '{{ csrf_token() }}', ... },
            body: JSON.stringify({ tipo, decision: this.licDecision })
        });
        const data = await resp.json();
        this.licEstado = data.licencia_estado; // Actualiza la UI reactivamente
        this.loading = false;
    }
}">
    <!-- :class vincula clases CSS al estado reactivo -->
    <span class="badge" :class="licEstado === 'valida' ? 'bg-success' : 'bg-seco
          x-text="licEstado"></span>

    <button :disabled="loading" @click="verificar('licencia')">
```

```
        <span x-show="loading" class="spinner-border spinner-border-sm"></span>
        Confirmar
    </button>
</div>
```

> 💡 **Alpine vs React/Vue:** Alpine.js es ideal para añadir "destellos" de interactividad a páginas que ya genera el servidor (como las de Laravel). No requiere compilación ni `node_modules`. React o Vue serían excesivos para este proyecto.

Vite — El bundler de assets

Vite compila y optimiza los archivos JavaScript y CSS del proyecto. En desarrollo (`npm run dev`) proporciona Hot Module Replacement (los cambios se reflejan en el navegador al instante sin recargar).

Seeders y Factories: datos de prueba

12

Cómo poblamos la base de datos para desarrollo y testing

¿Qué son los seeders?

Los **seeders** son clases PHP que insertan datos en la base de datos. Son fundamentales en desarrollo para tener datos con los que trabajar sin tener que introducirlos manualmente.

```
// database/seeders/DatabaseSeeder.php — Seeder principal
public function run(): void
{
    // Crea el usuario admin con credenciales conocidas
    User::create([
        'name'     => 'admin',
        'email'    => 'admin@escalada.com',
        'password' => bcrypt('admin'),   // bcrypt() hace el hash de la contrase
        'rol'      => 'admin',
        'dni'      => '00000000A',
        'fecha_nacimiento' => '1990-01-01',
        'provincia' => 'Sevilla',
        'talla'    => 'M',
        'genero'   => 'otro',
    ]);

    // Crea 2 copas de competición
    $copaBloque = Copa::create([
        'name' => 'Copa de Bloque 2026', 'tipo' => 'Bloque', 'temporada' => 202
    ]);

    // Crea ubicaciones usando una Factory (genera datos realistas aleatoriament
    $ubicaciones = [];
    foreach (['Sevilla', 'Málaga', 'Granada', ...] as $prov) {
        $ubicaciones[$prov] = Ubicacion::factory()->create(['provincia' => $prov
    }

    // Crea 7 competiciones de ejemplo
    Competicion::create([
        'name'              => '1ª Prueba de Bloque de Andalucía, Sevilla',
        'tipo'              => 'bloque',
        'fecha_realizacion' => '2026-02-14 10:00:00',
        'copa_id'           => $copaBloque->id,
        'ubicacion_id'      => $ubicaciones['Sevilla']->id,
        ...
    ]);

    // Crea 300 usuarios aleatorios con la factory
```

```
    User::factory()->count(300)->create();

    // Ejecuta sub-seeders
    $this->call(TestUsersSeeder::class);     // Crea usuarios de test con roles
    $this->call(InscripcionesSeeder::class); // Inscribe competidores automática
}
```

Factories: datos realistas con Faker

Las **factories** usan la librería **Faker** para generar datos aleatorios pero realistas (nombres, emails, fechas, etc.):

```
// database/factories/UserFactory.php
class UserFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name'             => fake()->name(),            // "María García Lo
            'email'            => fake()->unique()->email(), // "maria@example.c
            'password'         => static::$password ??= Hash::make('password'),
            'dni'              => strtoupper(fake()->unique()->bothify('########
            'fecha_nacimiento' => fake()->dateTimeBetween('-30 years', '-15 year
            'provincia'        => fake()->randomElement(['Sevilla', 'Málaga', 'C
            'talla'            => fake()->randomElement(['XS', 'S', 'M', 'L', 'X
            'genero'           => fake()->randomElement(['M', 'F', 'otro']),
            'rol'              => 'competidor',
        ];
    }
}
```

InscripcionesSeeder: simulación de datos realistas

Este seeder es especialmente complejo: inscribe 150 competidores en cada competición con una distribución de estados que simula la realidad:

Estado	Probabilidad	Descripción
Pendiente (sin docs)	30%	Inscrito pero no ha subido documentos
Aprobada	30%	Ambos documentos verificados y válidos
Rechazada (licencia)	20%	La licencia federativa no es válida
Rechazada (pago)	10%	El justificante de pago no es válido
Pendiente (lic OK)	10%	Licencia verificada, esperando verificar pago

Comandos de base de datos

```
php artisan migrate                 # Ejecuta migraciones pendientes
php artisan migrate:fresh           # Borra todas las tablas y re-migra
php artisan migrate:fresh --seed    # Borra, re-migra y ejecuta todos los seede
php artisan db:seed                 # Ejecuta todos los seeders (sin borrar)
php artisan db:seed --class=TestUsersSeeder  # Ejecuta un seeder específico
php artisan migrate:status          # Muestra el estado de todas las migracione
```

Diagrama completo de la aplicación

Una vista de pájaro de toda la arquitectura

13

Mapa de rutas y accesos por rol

URL	Método	Controlador@método	Rol mínimo	Descripción
/dashboard	GET	Closure (web.php)	auth	Redirige según rol
/admin/pruebas	GET	Admin@pruebas	admin	Lista competiciones
/admin/copas	GET	Admin@copas	admin	Lista copas
/admin/copas	POST	Copa@store	admin	Crea copa
/admin/copas/{copa}	PATCH	Copa@update	admin	Edita copa
/admin/copas/{copa}	DELETE	Copa@destroy	admin	Borra copa
/admin/usuarios	GET	Admin@usuarios	admin	Lista usuarios
/admin/usuarios/{user}/rol	PATCH	Admin@updateRol	admin	Cambia rol
/admin/rocodromos	GET	Admin@rocodromos	admin	Lista rocódromos
/admin/competiciones	POST	Competicion@store	admin	Crea competición
/admin/competiciones/{id}	PATCH	Competicion@update	admin	Edita competición
/admin/competiciones/{id}/arbitro	PATCH	Admin@asignarArbitro	admin	Asigna árbitro
/arbitro/	GET	Arbitro@panel	arbitro	Panel árbitro
/arbitro/entrenador	GET	Arbitro@panelEntrenador	arbitro	Gestión equipo

/arbitro/deportista	GET	Arbitro@panelDeportista	arbitro	Mis inscripciones
/arbitro/competicion/{id}	GET	Arbitro@competicion	arbitro	Categorías de la competición
/arbitro/inscripcion/{id}/validar	PATCH	Arbitro@validarLicencia	arbitro	Valida documento
/competiciones/{id}	GET	Inscripcion@show	auth	Detalle competición
/inscripciones/{id}	POST	Inscripcion@store	auth	Inscribirse
/inscripciones/{id}/licencia	POST	Inscripcion@uploadLicencia	auth	Sube licencia
/entrenador/solicitar	POST	Entrenador@solicitarVinculo	entrenador	Solicita vínculo
/entrenador/inscribir	POST	Entrenador@inscribir	entrenador	Inscribe a equipo

Relaciones entre modelos

```
                    ┌─────────┐
                    │  Copa   │
                    │---------│
                    │ name    │
                    │ tipo    │
                    │temporada│
                    └─────┬───┘
                          │ hasMany
              ┌───────────▼──────────┐
              │    Competicion       │
              │----------------------│      ┌──────────┐
┌─────────────┤ copa_id (nullable)   ├──────┤ Ubicacion│
│             │ arbitro_id (nullable)│      └──────────┘
│             │ fecha_realizacion    │
│             │ fecha_fin            │
│             │ tipo, campeonato     │
│             │ categorias (JSON)    │
│             └──────────┬───────────┘
│                        │ hasMany
│                 ┌──────▼──────┐
│                 │ Inscripcion │
│                 │-------------│
│                 │ user_id     │◄──────────┐
│                 │ estado      │           │
│                 │ licencia_*  │           │
│                 │ pago_*      │           │
```

```
    |                 └──────────────┘            |
    |                                             |
    |                 ┌───────────────────────────┐
    |                 |        User               |
    |                 |---------------------------|
    └────────────────┤ rol (admin/arb/ent/comp)|
     belongsTo       | dni, genero, talla        |
      (árbitro)      | fecha_nacimiento          |
                      └───────────────────────────┘

                              | M:N (entrenador_competidor)
                              └──────────────────────────────▶ User (competidores)
```

Comandos de desarrollo habituales

```
composer run dev          # Arranca servidor + queue + Vite (todo en uno)
php artisan serve         # Solo el servidor web (http://localhost:8000)
npm run dev               # Solo Vite (HMR para CSS/JS)
php artisan tinker        # REPL interactivo para probar código Laravel
php artisan route:list    # Lista todas las rutas registradas
php artisan make:model    # Crea un nuevo modelo
php artisan make:controller  # Crea un nuevo controlador
php artisan make:migration   # Crea una nueva migración
```